

Herramientas y entorno de trabajo

Cómo usar el conjunto de herramientas (Python, sus librerías, Jupyter, LibreOffice y Git) para el proyecto «Análisis del Impacto de Hábitos de Vida y Factores Demográficos en los Gastos Médicos».

Documento técnico-didáctico. Escrito para que **una persona no experta o una IA colaboradora** entiendan qué hace cada herramienta, por qué se usa y cómo se conecta con el resto. Las afirmaciones técnicas están contrastadas con la documentación oficial (ver Referencias).

0. Mapa mental: ¿qué herramienta hace qué?

Antes de los detalles, la foto completa. Un proyecto de minería de datos es una **cadena de montaje**: cada herramienta resuelve una etapa.

Etapa	Herramienta	Para qué sirve aquí
Guardar y compartir código	Git + GitHub	Que todo el equipo trabaje sobre la misma base sin pisarse.
Entorno reproducible	Python + venv + requirements.txt	Que el proyecto corra igual en cualquier computadora.
Cuaderno de análisis	Jupyter Notebook	Escribir código y ver resultados/gráficas paso a paso.
Manejo de datos	pandas + NumPy	Cargar el CSV, limpiarlo, calcular estadísticas.
Visualización	Matplotlib + seaborn	Histogramas, dispersión, mapa de calor de correlación.
Modelado	scikit-learn + statsmodels	Regresión lineal y sus métricas.

1. Python y el entorno reproducible

Python es el lenguaje que usaremos. Lo importante no es solo «tener Python», sino tener un **entorno reproducible**: la misma versión de cada librería en todas las máquinas. Eso evita el clásico «en mi compu sí funciona».

Entorno virtual (venv)

Un *entorno virtual* es una carpeta aislada con su propia copia de Python y librerías, separada del sistema. Así, lo que instala este proyecto no rompe otros.

```
python3 -m venv .venv          # crea el entorno (una sola vez)
source .venv/bin/activate      # actívalo (macOS/Linux)
.venv\Scripts\activate        # actívalo (Windows)
pip install -r requirements.txt # instala TODAS las librerías del proyecto
```

El archivo requirements.txt

Es la «lista de ingredientes» con versiones exactas (ej. `pandas==3.0.3`). El operador `==` fija la versión para que el análisis sea **reproducible**: cualquiera obtiene los mismos resultados. Este proyecto ya incluye `pandas`, `numpy`, `matplotlib`, `seaborn`, `scikit-learn`, `statsmodels` y las librerías para generar documentos (`python-docx`, `python-pptx`, `openpyxl`).

Buena práctica. Cuando instales una librería nueva, agrégala a `requirements.txt` (o corre `pip freeze > requirements.txt`). Si no, tus compañeros no la tendrán.

2. Jupyter Notebook: el cuaderno de laboratorio

Un **notebook** (`.ipynb`) mezcla *celdas de código* y *celdas de texto*. Ejecutas una celda y ves el resultado justo debajo: tablas, números y gráficas. Es ideal para el Análisis Exploratorio porque documenta el razonamiento paso a paso, tal como pide la rúbrica («pantallazos del código y resultados»).

```
jupyter notebook          # abre la interfaz en el navegador
# o desde VS Code: abrir el archivo .ipynb directamente
```

Cuidado con el orden. Un notebook recuerda el estado de las variables. Si ejecutas celdas fuera de orden puedes engañarte. Antes de entregar, usa «*Restart & Run All*» para confirmar que todo corre de arriba a abajo sin errores.

3. pandas y NumPy: el corazón del manejo de datos

NumPy aporta los arreglos numéricos rápidos; **pandas** se construye encima y ofrece el **DataFrame**: una tabla con filas y columnas, como una hoja de Excel pero programable.

Operaciones que usarás sí o sí

Código	Qué hace
<code>pd.read_csv("data/insurance.csv")</code>	Carga el dataset en un DataFrame.
<code>df.head()</code>	Muestra las primeras filas para inspección.
<code>df.info()</code>	Tipos de dato y conteo de no-nulos por columna.
<code>df.describe()</code>	Estadística descriptiva (media, desviación, cuartiles).
<code>df.isnull().sum()</code>	Cuenta valores faltantes por columna.
<code>df.duplicated().sum()</code>	Cuenta filas duplicadas.
<code>df.drop_duplicates()</code>	Elimina duplicados.
<code>df.groupby("smoker") ["charges"].mean()</code>	Gasto medio por grupo (fumador vs no).
<code>df.corr(numeric_only=True)</code>	Matriz de correlación entre variables numéricas.

Dato verificado. En este dataset (1 338 filas), pandas detecta exactamente **1 fila**

duplicada y 0 nulos; tras `drop_duplicates()` quedan **1 337** registros. (Comprobado ejecutando el propio dataset del repositorio.)

4. Matplotlib y seaborn: visualización

Matplotlib es la librería base de gráficas; **seaborn** se apoya en ella y produce gráficos estadísticos bonitos con poco código. Para este proyecto, las gráficas clave son:

Gráfica	Función	Qué revela
Histograma	<code>sns.histplot(df["charges"])</code>	La distribución de gastos (aquí, muy sesgada a la derecha).
Dispersión	<code>sns.scatterplot(x="bmi", y="charges", hue="smoker", data=df)</code>	Relación IMC–gasto, separando fumadores por color (interacción clave).
Caja (boxplot)	<code>sns.boxplot(x="smoker", y="charges", data=df)</code>	Compara la mediana y los atípicos entre grupos.
Mapa de calor	<code>sns.heatmap(df.corr(numeric_only=True), annot=True)</code>	Matriz de correlación visual (ya está en el notebook base).

Consejo didáctico. Una gráfica sin título ni ejes etiquetados no comunica. Añade siempre `plt.title(...)`, `plt.xlabel(...)`, `plt.ylabel(...)`. Para guardar: `plt.savefig("reports/figures/nombre.png", dpi=150, bbox_inches="tight")`.

5. scikit-learn y statsmodels: el modelado

Ambas hacen regresión, pero con enfoques complementarios:

	scikit-learn	statsmodels
Enfoque	Machine learning / predicción	Estadística inferencial

Fortaleza	Entrenar, predecir, métricas, validación cruzada	Tabla con p-valores, intervalos de confianza, R^2 ajustado
Úsala para...	Medir qué tan bien predice el modelo	Interpretar si cada variable es estadísticamente significativa

```
# scikit-learn: entrenar y evaluar
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
modelo = LinearRegression().fit(X_train, y_train)
pred = modelo.predict(X_test)
print(r2_score(y_test, pred))

# statsmodels: interpretar (p-valores, significancia)
import statsmodels.api as sm
modelo_sm = sm.OLS(y, sm.add_constant(X)).fit()
print(modelo_sm.summary())
```

Las fórmulas y la interpretación de estas técnicas se explican a fondo en la **Investigación 2** de esta serie.

6. LibreOffice (modo headless): generar documentos

LibreOffice puede ejecutarse **sin abrir ventana** («headless») para convertir archivos entre formatos. El repositorio ya trae scripts que lo automatizan y guardan la salida en docs/generated_docs/.

```
bash scripts/setup_libreoffice.sh # instalar (una vez)
python scripts/generate_document.py reporte.html pdf # convertir a PDF
```

Regla verificada empíricamente. LibreOffice convierte *cualquier* formato a **PDF** de forma fiable. En cambio, **no** existe filtro para pasar HTML directo a .docx. Por eso: para crear Word/PowerPoint/Excel se generan con Python (python-docx, python-pptx, openpyxl) y LibreOffice se usa para llevarlos a PDF. La guía completa está en

scripts/README.md.

7. Git y GitHub: trabajo colaborativo

Git lleva el historial de cambios; **GitHub** lo aloja en la nube. Flujo mínimo para el equipo:

```
git checkout -b mi-analisis      # crear una rama propia
# ...editar archivos...
git add .                        # marcar cambios
git commit -m "Agrega análisis de regresión"
git push origin mi-analisis      # subir a GitHub
# luego abrir un Pull Request para integrar a main
```

Regla de oro. No trabajen todos directo sobre main. Cada quien en su rama y se integra con Pull Request; así se revisa y no se pierde trabajo.

8. Errores comunes (y cómo evitarlos)

Síntoma	Causa probable	Solución
ModuleNotFoundError	Librería no instalada / entorno sin activar	<code>source .venv/bin/activate</code> y <code>pip install -r requirements.txt</code>
Ruta del CSV no encontrada	Rutas relativas dependen de dónde corres	Correr desde la raíz del repo, o usar rutas claras (<code>../data/insurance.csv</code> desde el notebook)
Resultados que «cambian solos»	Celdas ejecutadas fuera de orden	«Restart & Run All» antes de concluir
Gráfica ilegible	Sin títulos ni etiquetas	Añadir <code>title</code> , <code>xlabel</code> , <code>ylabel</code>

9. Cómo lo haría una IA (resumen operativo)

1. Activar el entorno e instalar dependencias (`pip install -r requirements.txt`).
2. Cargar el CSV con pandas y verificar tipos, nulos y duplicados.
3. Explorar con seaborn/matplotlib (distribuciones, correlación).
4. Modelar con scikit-learn (predicción) y confirmar significancia con statsmodels.
5. Redactar el reporte en HTML y convertirlo a PDF con `scripts/generate_document.py`.

Referencias

1. McKinney, W. *pandas — Documentación oficial*. pandas.pydata.org/docs
2. Harris, C. R. et al. (2020). «Array programming with NumPy». *Nature* 585, 357–362. numpy.org/doc
3. Hunter, J. D. (2007). «Matplotlib: A 2D Graphics Environment». *Computing in Science & Engineering*. matplotlib.org
4. Waskom, M. (2021). «seaborn: statistical data visualization». *JOSS*. seaborn.pydata.org
5. Pedregosa, F. et al. (2011). «Scikit-learn: Machine Learning in Python». *JMLR* 12, 2825–2830. scikit-learn.org
6. Seabold, S. & Perktold, J. (2010). «statsmodels». statsmodels.org
7. Project Jupyter. docs.jupyter.org · The Document Foundation. *LibreOffice*. libreoffice.org
8. VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly. [edición en línea gratuita](#)